

Rewriting Ontology-Mediated Navigational Queries into Cypher

36th International Workshop on Description Logics
September 2, 2023, Rhodes, Greece

Nikola Dragovic¹

Cem Okulmus²

Magdalena Ortiz^{1,2}

¹ TU Wien, Austria

² Umeå University, Sweden

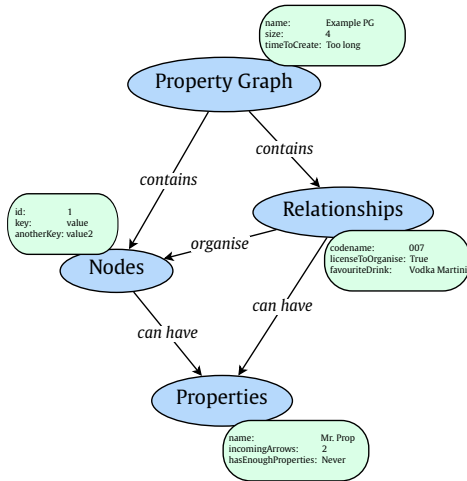


UMEÅ UNIVERSITY



Graph Database Systems and Ontology-Mediated Queries

- Graph Database Systems (GDBS) use a different data model than classical relational database systems: **property graphs**.



Graph Database Systems and Ontology-Mediated Queries

- Graph Database Systems (GDBS) use a different data model than classical relational database systems: **property graphs**.
- Ontologies allow to **extend the data** with external knowledge:
 - ▶ All companies must have an owner (*existential requirements*)
 - ▶ Every mother & every father is a parent (*concept inclusions*)
 - ▶ And more ... (depending on the expressiveness of the ontology)
- Ontology-Mediated Queries (OMQs) are queries that incorporate the ontological knowledge by some means. One such means lies in the **rewriting** of the query.

Graph Database Systems and Ontology-Mediated Queries

- Graph Database Systems (GDBS) use a different data model than classical relational database systems: **property graphs**.
- Ontologies allow to **extend the data** with external knowledge:
 - ▶ All companies must have an owner (*existential requirements*)
 - ▶ Every mother & every father is a parent (*concept inclusions*)
 - ▶ And more ... (depending on the expressiveness of the ontology)
- Ontology-Mediated Queries (OMQs) are queries that incorporate the ontological knowledge by some means. One such means lies in the **rewriting** of the query.

OMQs shown great use in relational setting, but one big issue prevented their use in GDBS ...

Navigational Queries: C2RPQs

- Navigational queries allow to express reachability and find certain connection (or paths) between any nodes in the data.
 - ▶ This is not expressible in SQL, without the use of recursive views
- Such queries are well known in literature in form of C2RPQs

Conjunctive 2-way Regular Path Queries (C2RPQs)

Of the form $q(\vec{x}) = \exists \vec{y}. \varphi$, where $\vec{x}, \vec{y}$ are disjoint tuples of variables, and φ is a conjunction of atoms of the form:

- $A(t)$, where A is a unary concept and $t \in N_I \cup \vec{x} \cup \vec{y}$.
- $E(t, t')$, where E is a regular expression and $t, t' \in N_I \cup \vec{x} \cup \vec{y}$ and E can use (inverted) relationships.

We denote by N_I the set of nodes in the property graph.

Sadly, the most used GDBS, Neo4J, does not support C2RPQs in its query language Cypher.

Cypher and Trail Semantics

- We focus on Neo4J and its query language Cypher.
- Crucial restriction: path that are matched **need to be trails**, i.e., no repeated edges are allowed.
- Trails and C2RPQs do not go well together: evaluation of C2RPQs under trail semantics is NP-complete.
- Other restriction: Cypher does not allow mixing directions: all relationships occurring in a Cypher query must point to the same direction, all forward or all inverted.

Cypher and Trail Semantics

- We focus on Neo4J and its query language Cypher.
- Crucial restriction: path that are matched **need to be trails**, i.e., no repeated edges are allowed.
- Trails and C2RPQs do not go well together: evaluation of C2RPQs under trail semantics is NP-complete.
- Other restriction: Cypher does not allow mixing directions: all relationships occurring in a Cypher query must point to the same direction, all forward or all inverted.

Problem:

The existing work on OBDA and C2RPQs cannot be directly leveraged in Neo4J, due to differences highlighted above.

Our Solution: NPGQs and Rewritings

Navigational Property Graph Queries (NPGQs)

Of the form $q(\vec{x}) = \exists \vec{y}. \phi(\vec{x}, \vec{y})$, where ϕ is a conjunction of the form:

$$T(x) \quad T(x, y) \quad (A_1 \cup \dots \cup A_n)(x) \quad (r_1 \cup \dots \cup r_n)(x, y) \quad (r_1 \cup \dots \cup r_n)^*(x, y)$$

where each A_i represents set of nodes, each r_i a relationship, T is a data test, and $x, y \in \vec{x} \cup \vec{y} \cup N_I$. An NPGQ is *pure* if it contains no r_i atoms in last two columns that mix directionality.

Path and Trail Answers

For a PG $\mathcal{G}$, an NPGQ $q(\vec{x})$ and a mapping π from individuals and variables in $q(\vec{x})$ to nodes in $\mathcal{G}$, we call π a path match for $q(\vec{x})$ in $\mathcal{G}$ if

1. All data tests $T(x)$ and $T(x, y)$ are satisfied;
2. $\pi(c) = c$ for each $c \in N_I$;
3. for each atom $(A_1 \cup \dots \cup A_n)(x)$ in $q(\vec{x})$; $\{A_1, \dots, A_n\} \cap c_I(\pi(x)) \neq \emptyset$;
4. For each $(r_1 \cup \dots \cup r_n)(x, y)$ or $(r_1 \cup \dots \cup r_n)^*(x, y)$, there exists a path p from $\pi(x)$ to $\pi(y)$ where p in $\mathcal{G}$ only follows the resp. relationships from the union.

If additionally the path p in item 4 is a trail, we call π a trail match. A tuple $\vec{a} \subseteq N_I$ is a (*trail*) *answer* to a query $q(\vec{x})$ if there exists a (*trail*) match π for $\vec{x}$ s.t. $\pi(\vec{x}) = \vec{a}$.

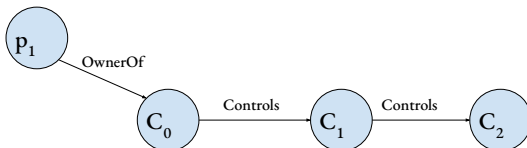
Our Solution: NPGQs and Rewritings (cont.)

Example NPGQs

We have a property graph of a national bank, containing the various financial entities like companies, trusts and also contains private individuals, who can act as investors, owners or officials of these financial entities.

Let us imagine we are interested in finding all owners, who own a high revenue company indirectly, meaning via intermediate companies.

$\text{OwnIndirect}(x, y) \leftarrow \text{OwnerOf}(x, z), \text{Controls}^*(z, y), \text{revenue} \geq 5000(y).$



Our Solution: NPGQs and Rewritings (cont.)

- By construction, path and trail answers for NPGQ coincide

Theorem

Let $\mathcal{G}$ be a property graph and q an NPGQ. Then the answers to q over $\mathcal{G}$ under trail semantics are also path answers.

Proof (Sketch)

The basic observation lies in the fact that for any path that witnesses a path match, but is not a trail, there must exist a trail with the same source and target nodes, as we can simply drop the repeated edges (which form a cycle) and still reach the same target: hence there also exists a trail and thus a trail match.

- Note: This property does not hold for C2RPQs! It requires the restrictions of NPGQs.

Our Solution: NPGQs and Rewritings (cont.)

- NPGQs makes use of a property in Cypher to elegantly capture concept inclusions hierarchies from an ontology:

$$\mathcal{T} = \{A \sqsubseteq B, C \sqsubseteq B\}$$



$$q^{\mathcal{T}}(x) = (A \cup B \cup C)(x)$$

$$q(x) = B(x)$$

- Such **concept unions** can compactly express what would otherwise require (in classical OBDA) a union of queries, representing every possible rewriting of q with respect to $\mathcal{T}$.

Implementation: Rewriting NPGQ & Ontology to Cypher

- To show the viability of our contribution, we also worked on a proof-of-concept implementation.

<https://github.com/nikdra/omq-pg>

- Takes as input an OWL 2 QL ontology, and an NPGQ, and outputs a Cypher query that produces the same output on the data, as the NPGQ would when considering the ontology.

Real-World Benefits

- Together with industrial partner, we developed 5 representative queries for two publicly available data sets¹
- We use an ontology to allow querying both datasets
- We show that our rewriting scales very well with query size
 - ▶ The comparison is against a naive rewriting that is comparable to PerfectRef (which does not use concept unions).

Query	Union Rewr. size	Output size	Exec. time for Union Rewr. (sec)	PerfectRef* rewr. size	Exec. time for PerfectRef* (sec)
Q1	1	234	0.039	8	0.001
Q2	33	751	0.705	880	111.071
Q3	33	264	0.538	880	109.209
Q4	11	10	0.195	1056	136.767
Q5	11	8	0.222	1056	143.516

¹<https://www.nuscenes.org/>
<https://woven-planet.github.io/l5kit/dataset.html>

Conclusion & Future Work

- First practical steps towards OMQs for Neo4J and its query language Cypher. Real-world benefits already readily apparent.

Future work

- Focus on more than just Cypher: **GQL**
 - ▶ This might allow us to capture a larger fragment of C2RPQs
 - ▶ Real-world use hinges on systems following the new standard
- Novel rewriting can help classic OBDA: **'23 SQL extension**
 - ▶ Could the concept unions we use in our rewriting also benefit relational data bases that support property graph queries?
- Longer term goal: enabling OBDA over both graph database systems and relational database systems, thus enabling transparent access to both via an ontology layer

Thank You For Your Attention!



UMEÅ UNIVERSITY

